# Operations: Deployment, Monitoring & Rollback

Operational runbook for the OptimaTech LMS production stack (KAN-60, KAN-147).

## 1. Deploy targets

| Component | Platform | Build → Deploy | Health / liveness |
|---|---|---|---|
| `backend/` (NestJS API) | Railway | `npm run build` → `npm run start:prod` | `GET /api/v1/health` |
| `web/` (Next.js, OpenNext) | Cloudflare Workers | `npm run deploy` (`opennextjs-cloudflare build && … deploy`) | Worker `optima-lms-web` |
| `functions/` (Firebase) | Firebase Functions | `firebase deploy --only functions` | Firebase console |

Railway deploy config lives in `backend/railway.json`. Cloudflare config lives in `web/wrangler.jsonc` (worker `optima-lms-web`, observability logs + traces enabled).

### `backend/railway.json`, field by field

| Field | Value | Why |
|---|---|---|
| `healthcheckPath` | `/api/v1/health` | **Liveness, not readiness.** Railway gates activation of a new deployment on this returning 200 — that is the zero-downtime switch. It must stay dependency-free: pointing it at `/health/ready` would let a transient Redis blip fail the healthcheck and roll back a good deploy. |
| `healthcheckTimeout` | `300` | Seconds Railway waits for the first 200 before declaring the deploy failed. |
| `overlapSeconds` | `20` | Seconds the previous deployment keeps serving after the new one goes active — covers in-flight requests and clients still holding a connection to the old instance. |
| `drainingSeconds` | `30` | Seconds between SIGTERM and SIGKILL on the old deployment. Must exceed the app's own shutdown budget (§2.4), or the platform kills it mid-cleanup. |
| `numReplicas` | `1` | Left at 1 for cost. Raising it is **safe as of KAN-147**: rate-limit counters live in Redis, the transcoder writes only to an ephemeral temp dir, and the quiz-expiry cron takes a Redis lock so it still runs once per tick. |
| `restartPolicyType` / `MaxRetries` | `ON_FAILURE` / 5 | Crash-loop containment. |

> Railway's **service settings override this file**. If a value here does not seem to apply, check Service → Settings for a UI value set previously.

### CI / CD pipelines

| Workflow | Trigger | What it does |
|---|---|---|
| `.github/workflows/ci.yml` | every PR + push to `main` | Per-package lint / type-check / build / test. On a PR only the packages the diff touches run; `main` always runs |

| Workflow | Trigger | What it does |
|---|---|---|
| | | everything. Backend publishes a coverage summary + artifact. The aggregate `CI` job is the one to require in branch protection — the package jobs are skipped (not passed) on unrelated PRs, which would otherwise stall a required check. |
| `.github/workflows/security.yml` | every PR + push to `main` + weekly (Mon 06:00 UTC) | Secret scan (gitleaks — **blocking** on PRs over the commits the PR adds, informational full-history sweep otherwise; see §6), dependency review of what a PR adds, and `npm audit` per package. |
| `.github/workflows/deploy-web.yml` | **manual only** (`workflow_dispatch`) | Reproducible Cloudflare deploy of `web/`. Runs in the `production` GitHub Environment, then health-checks the result. |
| `.github/workflows/deploy-backend.yml` | **manual only** (`workflow_dispatch`) | Lint + build + test, then `railway up` into the `production` Environment, then verifies **liveness and readiness** before reporting success. |
| `.github/dependabot.yml` | weekly (Mon) | Grouped dependency PRs for the three npm packages, Flutter `pub`, and the GitHub Actions themselves. |

**Audit gate:** `npm audit` blocks only on **critical**. The measured baseline was 0 critical but 25-28 **high** per package (mostly transitive dev tooling), so gating on `high` would fail every PR on pre-existing debt. Raise the threshold in `security.yml` once Dependabot has worked the backlog down.

### The manual approval gate

Both deploy workflows declare `environment: production`. A GitHub Environment with **required reviewers** is what turns that line into an approval gate: the job pauses before its first step and waits for a named human to approve.

Until that Environment exists, both workflows run unprotected — the declaration is inert, not enforcing.

> **The backend gate is not real until Railway's auto-deploy is off.** Railway deploys `main` on push by default, which bypasses GitHub Actions entirely. Turn it off in Railway → the backend service → **Settings → Source → disable "Deploy on push"**, then add the `push:` trigger commented at the top of `deploy-backend.yml`. Do it in that order: adding the trigger first means every merge deploys twice.

**One-time setup to finish the CD half** — see §5 for the full click-by-click list.

## 2. Monitoring

### 2.1 Health endpoints

| Endpoint | Answers | Checks | Used by |
|---|---|---|---|
| `GET /api/v1/health` | "is the process alive?" | nothing | Railway `healthcheckPath`, uptime monitor |
| `GET /api/v1/health/ready` | "can this instance serve traffic?" | Redis + Data Connect | deploy verification, degraded-state monitor |

**Liveness** → `{ status, timestamp, uptime, env }`, always 200 while the process is running. Dependency-free on purpose: if it went red on a transient Redis blip, Railway would restart a healthy container and turn a 30-second dependency outage into a deploy loop.

**Readiness** → `200` when every dependency answers, `503` otherwise:

```
{
  "status": "ok",
  "timestamp": "2026-07-28T01:00:00.000Z",
  "checks": {
    "redis":       { "status": "up", "latencyMs": 2 },
    "dataconnect": { "status": "up", "latencyMs": 88 }
  }
}
```

A failing check reports `{"status":"down","latencyMs":3000,"error":"timeout"}` — `timeout` or `unreachable`, never the driver's message. The endpoint is unauthenticated and raw driver errors carry internal hosts and ports; the full error goes to the server log instead.

While the process is shutting down it returns `503 {"status":"shutting_down"}` before running any check (§2.4).

Design points worth knowing before changing it:

- **Never wire readiness to `healthcheckPath`.** See §1.
- Both routes skip the global rate limiter. Beyond probe traffic sharing an IP bucket with real users, the fail-closed throttler would otherwise return *its own* 503 during a Redis outage — and readiness would report a generic failure instead of naming Redis, exactly when that answer matters most.
- Results are cached for `HEALTH_CACHE_MS` (10s) and concurrent probes share one in-flight run, so a load balancer, an uptime monitor and a deploy loop hitting it at once cost one Data Connect query, not three.
- Each check is timeout-raced at `HEALTH_CHECK_TIMEOUT_MS` (3s), well under the Data Connect client's own 15s budget.
- Neither route appears in the access log, and neither counts toward the error-rate alarm — a probe every few seconds would otherwise dominate the window and dilute a real spike into nothing. Readiness state changes are logged once per **transition**, not once per probe.

## 2.2 Alerting

Configured by a single variable: **`ALERT_WEBHOOK_URL`**. When it is unset the alerter no-ops and says so once at startup — local and CI runs stay silent. The payload is Slack-shaped (`{"text": …}`), which Slack Incoming Webhooks, Discord's `/slack`-suffixed endpoint and Teams workflow URLs all accept unchanged.

| What fires | Severity | Dedupe key | Tuning |
|---|---|---|---|
| Any 5xx from the exception filter | critical | `5xx:<method> <path>:<ErrorClass>` | `ALERT_COOLDOWN_MS` (5m) |
| 5xx share of traffic over threshold | critical | `error-rate` | `ALERT_ERROR_RATE_THRESHOLD` (0.1), `_WINDOW_MS` (5m), `_MIN_SAMPLES` (20) |
| Readiness ok → failing | critical | `readiness` | edge-triggered |
| Readiness failing → ok | info | `readiness-recovered` | edge-triggered |
| Backend deploy workflow failed | — | — | posted by `deploy-backend.yml` |

Three deliberate properties:

- **Alerts are deduplicated per key**, not per occurrence. A bad deploy throwing the same error 4 000 times sends one message, and the next one after the cooldown carries a `(N more suppressed)` tally. The key uses the error *class*, not its message — messages embed ids, which would defeat the cooldown.

- **The alerter never blocks or throws.** The webhook POST is fire-and-forget with a 5s timeout; a failed alert is logged and dropped, never retried. Retrying would queue work during exactly the incident that broke it.
- `ALERT_ERROR_RATE_MIN_SAMPLES` **is what stops false pages.** Without it, the first request of a quiet night failing reads as a 100% error rate.

Alerts are also always written to the log ( `ALERT …` ), so the drain and the channel tell the same story.

> **What this cannot catch: total outage.** A process that is gone cannot alert about itself, and neither can a crash-looping container. That is the external uptime monitor's job — §2.3. Set both up, or you have alerting that goes quiet precisely when things are worst.

## 2.3 External uptime monitoring

**Monitors to create** (both, on the same alert contact):

| Monitor | URL | Expect | Meaning if it fires |
|---|---|---|---|
| API liveness | `https://learning-management-system-production-a97c.up.railway.app/api/v1/health` | HTTP 200, keyword `"status":"ok"` | The API is down — page. |
| Web | `https://optima-lms-web.it-e0e.workers.dev/` | HTTP 200 | The web app is down. |
| API readiness *(optional)* | `https://learning-management-system-production-a97c.up.railway.app/api/v1/health/ready` | HTTP 200 | Degraded: process alive, a dependency is not. Route this to a non-paging channel — in-app alerting already covers it. |

**UptimeRobot (free)** — Add New Monitor → HTTP(s) → paste URL → set the interval → under *Alert Contacts To Notify* tick your email → Create. Use **Keyword** type for the API monitor with keyword `"status":"ok"`, so a proxy returning a 200 error page still counts as down.

Two things to know about the free tier:

- The check interval is **5 minutes**, so worst-case detection lag is ~5 minutes. 60-second checks are on the paid Solo plan. (This runbook previously claimed a 1-minute check — it was never achievable on the free tier.)
- Since Dec 2024 the free plan is licensed for **personal, non-commercial use**. For a commercial deployment, **Better Stack** is the closer fit: free tier, 10 monitors, 3-minute checks, commercial use allowed. The endpoints and alert routing are identical either way — only the vendor changes.

## 2.4 Graceful shutdown (what makes a deploy zero-downtime)

Railway's `overlapSeconds` / `drainingSeconds` decide *when* the old instance is signalled and killed; what it does in between is the app's job. On SIGTERM ( `backend/src/main.ts` ):

1. readiness flips to 503 — the router stops choosing this instance;
2. it keeps serving for `SHUTDOWN_DRAIN_DELAY_MS` (5s) so requests already routed here finish;
3. `app.close()` stops accepting connections, waits for open ones, then runs every module's shutdown hook (Redis `QUIT`, BullMQ worker close);
4. exit — with a `SHUTDOWN_TIMEOUT_MS` (20s) hard cap so a stuck handle cannot outlive Railway's SIGKILL deadline.

The whole budget (5s + close, capped at 20s) sits inside `drainingSeconds: 30` . **If you raise `SHUTDOWN_*`, raise `drainingSeconds` to match.**

> Nest's own `enableShutdownHooks()` is deliberately **not** used: it wires signals straight to `app.close()`, which would begin tearing the app down during step 2 and skip the drain. `close()` runs the lifecycle hooks either way.

**Known gap:** a video transcode can run for minutes and will not finish inside any sane drain window. Those jobs are killed and recovered by BullMQ's stalled-job retry, which is safe because the processor writes only to an ephemeral temp dir and uploads results to Storage. Moving the transcoder to its own service is the real fix if this becomes common.

### 2.5 Backend logs

Two global hooks (registered as `APP_INTERCEPTOR` / `APP_FILTER` providers in `backend/src/app.module.ts`) emit one line per request/error. Both log the path with the **query string stripped** — query values carry credentials (the HLS `?kt=` content-key token, Paymob's `?hmac=` signature).

- **`LoggingInterceptor`** (`[HTTP]` context) — one line per completed request: `GET /api/v1/courses -> 200 +42ms` (method, path, status, latency). Health probes (`/api/v1/health`, `/api/v1/health/ready`) are skipped so they don't bury real traffic. A handler that builds its own 5xx instead of throwing (the `@Res()` payment routes) never reaches the filter, so those are logged here at `error` level.
- **`AllExceptionsFilter`** (`[ExceptionFilter]` context) — thrown failures:
  - `5xx` → `error` level **with stack trace** (the primary incident signal).
  - `4xx` → `warn` level, one concise line, no stack. The line carries the real reason (validation field errors included), read off the response body.

View them in the Railway dashboard → service → **Logs**, or `railway logs`. To isolate errors, grep the level/context (`[ExceptionFilter]`) or the status arrow — the literal text is `-> 500`, so grep `-> 5` (not `-> 5xx`).

> **These are plain text lines, not JSON.** A drain can grep them but cannot aggregate by field (no error-rate-per-route, no p95 latency), and there is no request-correlation id, so an `[ExceptionFilter]` stack cannot be tied back to its `[HTTP]` access line. **This is why alerting is computed in-process (§2.2) rather than from a log drain** — a drain cannot alert on fields that aren't there. JSON logs plus an `x-request-id` middleware are tracked as **KAN-138**; once they land, log-drain alert rules become an option alongside the in-app ones.

> **Adding an APM later (optional):** wire an error tracker (e.g. Sentry) inside `AllExceptionsFilter.catch` — it already receives every unhandled exception in one place, next to the `AlertService` call, and it resolves through DI so a client can be injected. No-op when its DSN env var is unset (same pattern as `AlertService` and `functions/src/utils/mailer.ts`), so local/dev runs are unaffected.

### 2.6 Web (Cloudflare)

`observability` is enabled in `wrangler.jsonc` (100% head sampling for logs, 1% for traces). View in the Cloudflare dashboard → Workers & Pages → `optima-lms-web` → **Observability / Logs**, or tail live with `npx wrangler tail optima-lms-web`.

Cloudflare Workers deploys are atomic and versioned — a new version goes live instantly with no restart window — so the zero-downtime work in §2.4 applies to the backend only.

## 3. Rollback

### Backend (Railway)

Railway retains previous deployments and supports one-click rollback.

1. Railway dashboard → the backend service → **Deployments**.
2. Find the last known-good deployment (green, pre-incident).
3. ⋯ → **Rollback to this deployment** (redeploys that exact image; no rebuild).
4. Confirm recovery:
   - `curl https://learning-management-system-production-a97c.up.railway.app/api/v1/health` returns `200` (process is back),
   - `curl -i https://learning-management-system-production-a97c.up.railway.app/api/v1/health/ready` returns `200` (its dependencies are reachable — this is the one that catches a bad credential or a Redis outage that a liveness check happily ignores),

- Railway logs are clean of `[ExceptionFilter] … -> 5` server errors,
- the alert channel shows a `Readiness recovered` message if one had fired.

CLI alternative: `railway redeploy` (re-runs current) or roll back from the dashboard; Railway does not roll back via a single CLI flag, so prefer the Deployments UI for a precise target.

### Web (Cloudflare Workers)

Workers keeps a version history.

```
cd web
npx wrangler deployments list          # find the last good <version-id>
npx wrangler rollback --message "revert to <version-id>" <version-id>
```

`wrangler rollback` with no id rolls back to the immediately previous version. Verify by loading the app and checking `wrangler tail optima-lms-web`.

### Firebase Functions

```
cd functions
# Functions have no built-in version rollback — redeploy the previous known-good
# commit:
git checkout <good-commit> -- src && npm run build && firebase deploy --only functions

# IMPORTANT: `git checkout <commit> -- src` also STAGES the reverted files. Undo
# that once the fix is deployed, or the next commit silently re-ships the
# rolled-back code:
git restore --staged --worktree src
```

## 4. Incident runbook (quick)

1. **Detect** — an alert in the on-call channel (§2.2), an uptime-monitor email (§2.3), or a `[ExceptionFilter] … -> 5` spike in Railway / Cloudflare logs.

2. **Triage by which signal fired** — they mean different things:

| Signal | What it means | First move |
|---|---|---|
| Uptime monitor down, no app alerts | Process gone or crash-looping | Railway → Deployments: is it restarting? Check logs for a boot failure. |
| `Readiness check failing` | Process alive, dependency is not | `curl /health/ready` — the body names Redis or Data Connect. |
| `Error rate above threshold` | Broad regression, usually the last deploy | Roll back (§3). |
| A single repeated 5xx alert | One broken route | Read the stack in Railway logs; the alert carries the route and error class. |

3. **Assess** — backend, web, or a shared dependency? `curl /api/v1/health` and `curl /api/v1/health/ready` separate "process down" from "dependency down" in one step.

4. **Mitigate** — if the last deploy caused it, **roll back** the affected component (§3). Rollback is faster than forward-fixing under pressure.

5. **Verify** — liveness `200`, readiness `200`, error lines gone, key flows (login, catalog, enroll) work.

6. **Follow up** — capture the stack trace, open a ticket, fix forward, redeploy.

## 5. One-time external setup

Everything below is done outside the repo. The code is already in place and degrades safely while these are pending — alerting no-ops, the approval gate is inert, the post-deploy checks skip.

### 5.1 GitHub — the approval gate

1. **Settings** → **Environments** → **New environment** → `production`.
2. Tick **Required reviewers**, add yourself (and a teammate if there is one). *This single step is what makes both deploy workflows pause for approval.*
3. Optionally set **Deployment branches** to `main` only.

### 5.2 GitHub — secrets and variables

Under **Settings** → **Secrets and variables** → **Actions**. Scope the secrets to the `production` environment where possible.

| Name | Kind | Used by | Value |
|---|---|---|---|
| `RAILWAY_TOKEN` | secret | `deploy-backend.yml` | Railway → Account Settings → Tokens → Create. Use a **project token** scoped to this project. |
| `RAILWAY_SERVICE` | variable | `deploy-backend.yml` | The backend service's name as shown in Railway. |
| `API_HEALTH_URL` | variable | `deploy-backend.yml` | `https://learning-management-system-production-a97c.up.railway.app/api/v1/health` |
| `API_READY_URL` | variable | `deploy-backend.yml` | `https://learning-management-system-production-a97c.up.railway.app/api/v1/health/ready` |
| `ALERT_WEBHOOK_URL` | secret | `deploy-backend.yml` | Same webhook as the app (§5.4). Optional — without it the failure notification is skipped. |
| `CLOUDFLARE_API_TOKEN`, `CLOUDFLARE_ACCOUNT_ID`, `NEXT_PUBLIC_FIREBASE_API_KEY` | secrets | `deploy-web.yml` | As before. |
| `NEXT_PUBLIC_API_URL`, other `NEXT_PUBLIC_*`, `WEB_HEALTH_URL` | variables | `deploy-web.yml` | As before. |

Both workflows fail fast naming exactly what is missing, so a wrong name shows up in seconds rather than 20 minutes into a build.

Also, under **Settings** → **Branches**, require the `CI` and `Security` checks on `main` in branch protection.

### 5.3 Railway — service configuration

1. **Variables** — add the new ones from `backend/.env.example`. At minimum set `ALERT_WEBHOOK_URL` and `ALERT_ENV_LABEL=production`. The `HEALTH_*` and `SHUTDOWN_*` defaults are fine as shipped.
2. Fix `API_PUBLIC_URL` — it is `http://localhost:3000` in the committed `.env` and must be the real Railway hostname, or HLS key URLs point at nothing.
3. **Settings** → **check** `overlapSeconds` / `drainingSeconds` are not overridden in the UI (UI values win over `railway.json`).

4. **After this ticket ships:** Settings → Source → **disable "Deploy on push"**, then uncomment the `push:` trigger in `deploy-backend.yml`. In that order.

### 5.4 Slack (or Discord / Teams) — the alert channel

1. Slack → **api.slack.com/apps** → **Create New App** → **From scratch** → pick the workspace.
2. **Incoming Webhooks** → **Activate** → **Add New Webhook to Workspace** → choose the channel (e.g. `#lms-alerts`) → copy the `https://hooks.slack.com/services/…` URL.
3. Paste it into the Railway variable `ALERT_WEBHOOK_URL` **and** the GitHub secret of the same name.
4. Verify: the startup log should no longer say `ALERT_WEBHOOK_URL is not set`.

*Discord:* copy the channel webhook URL and append `/slack`. *Teams:* use a workflow webhook URL. No code change either way.

### 5.5 UptimeRobot — external monitoring

1. Sign up, then **+ New monitor**.
2. **Monitor 1 — API:** type *Keyword*, URL `https://learning-management-system-production-a97c.up.railway.app/api/v1/health`, keyword `"status":"ok"`, alert contact = your email.
3. **Monitor 2 — Web:** type *HTTP(s)*, URL of the deployed worker.
4. *(Optional)* **Monitor 3 — readiness:** type *HTTP(s)*, `https://learning-management-system-production-a97c.up.railway.app/api/v1/health/ready`. Route it to a quieter contact — in-app alerting already covers dependency failures, so this is a backstop.
5. Note the free-tier caveats in §2.3 (5-minute checks; non-commercial licence).

### 5.6 Verify the whole chain

```
curl -i https://learning-management-system-production-a97c.up.railway.app/api/v1/health         # 200, {"status":"ok",…}
curl -i https://learning-management-system-production-a97c.up.railway.app/api/v1/health/ready   # 200, both checks "up"
```

Then run **Actions** → **Deploy Backend** → **Run workflow**: it should pause for approval, deploy, and pass both verification steps.

## 6. Open security item: committed service-account key

The first run of the secret scan (`.github/workflows/security.yml`) found a **Firebase Admin SDK service-account private key committed to git history**.

| | |
|---|---|
| File | `backend/firebase-service-account.json` |
| Commits | `f930c7d` ("nestjs init", 2026-06-19) and `6e4e39b` |
| Identity | `firebase-adminsdk-fbsvc@learning-management-syst-3cc3a.iam.gserviceaccount.com` |
| Project | `learning-management-syst-3cc3a` |

It is **no longer in the working tree** and `.gitignore` now covers `**/firebase-service-account*.json` — but git history is permanent, so anyone who can read the repository (or clone it, or read it if it ever becomes public) can still extract the key. An Admin SDK key **bypasses Firebase Security Rules**: it grants full read/write on the database and Storage and can mint tokens for any user.

### Remediation, in order

1. **Rotate the key first.** Google Cloud Console → IAM & Admin → Service Accounts → that account → Keys → delete the leaked key, then create a new one and update `GOOGLE_APPLICATION_CREDENTIALS` / the Railway variable. Rotation is the only step that actually

revokes access — the leaked copy may already have been cloned, so purging history alone fixes nothing.
2. **Check for abuse.** Review Cloud Audit Logs for that service account around and after 2026-06-19.
3. **Then purge from history** (optional but recommended), e.g. `git filter-repo --invert-paths --path backend/firebase-service-account.json`, followed by a force-push and a re-clone by every contributor. Coordinate this — it rewrites every commit hash.
4. **Re-run the Security workflow.** Once the key is gone from history the full-history sweep goes clean, and the `secrets` job can be made blocking in all events by deleting its `continue-on-error` line.

Reviewed non-findings (Firebase *client* config, which is public by design; test fixtures; a README badge) are allowlisted by fingerprint in `.gitleaksignore`, each with a stated reason. The service-account key is deliberately **not** allowlisted so it keeps showing up until it is genuinely dealt with.